

Обмен данными с прикладными решениями на платформе «1С:Предприятие» в формате EnterpriseData

Введение

Что такое формат EnterpriseData
Как обмениваться данными в формате EnterpriseData

О формате EnterpriseData

Объекты
Пример описания объекта
Как с помощью формата EnterpriseData обмениваться данными с конфигурациями
Учет номеров отправленных и полученных сообщений
Получение подтверждения (квитанции) от приложения ZZ
Полная картина обмена

Внимание! Данный функционал доступен в "Библиотеке стандартных подсистем", начиная с версии 2.3.1.62.

Введение

Данная статья описывает технологию интеграции с прикладными решениями на платформе «1С:Предприятие» с использованием формата EnterpriseData. Статья разделена на две логических части:

1. ЧТО такое формат EnterpriseData?
2. КАК с помощью формата EnterpriseData обмениваться данными с информационными базами?

Вкратце эти 2 пункта описаны ниже, а в самой статье – подробно.

Что такое формат EnterpriseData

Это формат, позволяющий описать объект информационной базы (контрагента, накладную и т.п.) или сообщить о факте удаления этого объекта. Ожидается, что конфигурация, получившая файл в формате EnterpriseData, отреагирует соответствующим образом – создаст у себя новые объекты и удалит те, которые в файле помечены как удаленные.

Как обмениваться данными в формате EnterpriseData

Помимо информации об объектах, файл в формате EnterpriseData содержит заголовок (секцию <Header>) со служебной информацией. По этой информации конфигурация сможет понять, от какого внешнего приложения получена информация и какую информацию нужно отправить приложению в ответ (данные о тех объектах, которые были обновлены в информационной базе со времени последней синхронизации с данным сторонним приложением).

Таким образом, обмен данными в формате EnterpriseData с конфигурацией – это обмен файлами. В ответ на полученный от внешнего приложения файл конфигурация обрабатывает его и создаст файл-ответ. Обмен файлами может происходить:

- через выделенный файловый каталог,
- через каталог FTP,
- через веб-сервис, развернутый на стороне информационной базы. Файл с данными передается как параметр веб-методов.

Примечание. Для двустороннего обмена данными между сторонним приложением и конфигурацией на стороне информационной базы должен быть сделан ряд настроек – стороннее приложение должно быть зарегистрировано в информационной базе, для него должен быть определен канал обмена (через файловый или FTP-каталог) и т.п. Но для случаев простой интеграции, когда достаточно только передавать информацию от стороннего приложения в информационную базу и обратной передачи данных из информационной базы в стороннее приложение не требуется (например, интеграция онлайн-магазина, передающего информацию о продажах в «1С:Бухгалтерию»), есть упрощенный вариант работы через веб-сервис, не требующий настроек на стороне информационной базы.

О формате EnterpriseData

Формат **ED** включают в себя описание бизнес-сущностей из различных областей хозяйственно-экономической деятельности предприятия, и является расширяемым. В дальнейших версиях формата **ED**, фирма «1С» будет добавлять в него описание новых сущностей и расширять существующие новыми полями. Номера версий формата ED состоят из X.Y.Z, где X.Y. – это Major версия, Z – это Minor версия.

Major версия изменяется:

X – в случае глобальной реструктуризации объектов формата **ED**;

Y – в случаях добавления новых или незначительной реструктуризации существующих объектов формата **ED**. Примером таких изменений может быть: добавление новых объектов формата **ED**; добавление свойств, требующих обязательное заполнение; изменение ключевых свойств объектов формата **ED**.

Minor версия изменяется:

В случае исправления ошибок и прочих незначительных изменениях, при которых сохраняется логика конвертации данных, разработанная для предыдущих minor версий формата **ED**. Примером таких изменений может быть исправление ошибки, изменения свойств объектов формата **ED**, добавление свойств, использование которых при конвертации данных не является обязательным.

Изменение Minor версии не приводит к потери совместимости с предыдущими minor версиями. Major версии не имеют обратной совместимости.

Внимание! Гарантированный срок поддержки актуальной версии составит 1 год с момента выхода новой версии формата **ED**. Поддержка версии формата **ED** в составе конфигураций, использующих библиотеку стандартных подсистем (далее «**БСП**»), не может быть прекращена раньше гарантированного срока. Решение о фактическом прекращении поддержки версии формата **ED** принимается исходя из анализа ее использования при обмене между существующими прикладными решениями. Например, версия формата **ED** 1.4 будет гарантированно поддерживаться еще 1 год после выпуска версии 1.5.

Объекты

Формат описывает представление объектов (документов или элементов справочников) в виде XML-файлов. Версия 1.0.1 содержит описание 94-х объектов из различных областей (финансы, производство, закупки и продажи, складские операции). Названия типов, как правило, хорошо понятны и не нуждаются в дополнительных объяснениях: например, «Документ.АктВыполненныхРабот» или «Справочник.Контрагенты». Как можно заметить, описание типов документов начинается с префикса «Документ.», элемента справочника – с префикса «Справочник.».

Каждый сложный тип (complexType) который наследует тип Object (содержит в описании инструкцию <xs:extension base="ns1:Object">), является описанием самостоятельного объекта. Каждый объект содержит в себе составной элемент под названием «КлючевыеСвойства»; в этом элементе хранятся свойства, по которым объект может быть идентифицирован в системе. Например, для контрагента это НаименованиеПолное, ИНН и КПП. Почти все объекты (кроме документов остатков) имеют среди ключевых свойств свойство «Ссылка»; это Global Unique Identifier (**GUID**), записанный в виде строки в формате {G1G2G3G4-G5G6-G7G8-G9G10-G11G12G13G14G15G16}, где Gx — значение соответствующего байта структуры в шестнадцатеричном представлении. Поле «Ссылка» однозначно идентифицирует объект в системе (является ее первичным ключом).

Формат также содержит описание типов вспомогательных объектов, большинство из них описывает:

- Коллекцию элементов (например, «Документ.АктВыполненныхРабот.Услуги»). Является коллекцией (<sequence>) вспомогательных элементов.
- Вспомогательный (несамостоятельный) объект, например «Документ.АктВыполненныхРабот.Услуги.Строка». Такой объект не имеет ценности сам по себе, он должен быть включен в базовый объект (в данном случае – в «Документ.АктВыполненныхРабот», конкретно – в коллекцию «Документ.АктВыполненныхРабот.Услуги»).

Пример описания объекта

Рассмотрим формат EnterpriseData на примере сущности «Документ.АктВыполненныхРабот».

```
<xs:complexType name="Документ.АктВыполненныхРабот">
  <xs:complexContent>
    <xs:extension base="ns1:Object">
      <xs:sequence>
        <xs:element name="КлючевыеСвойства" type="tns:КлючевыеСвойстваАктВыполненныхРабот"/>
        <xs:element name="Ответственный" type="tns:КлючевыеСвойстваПользователь" minOccurs="0"/>
        <xs:element name="Валюта" type="tns:КлючевыеСвойстваВалюта"/>
        <xs:element name="Сумма" type="tns:ТипСумма"/>
        <xs:element name="СуммаВключаетНДС" type="xs:boolean"/>
        <xs:element name="Подразделение" type="tns:КлючевыеСвойстваПодразделение" minOccurs="0"/>
        <xs:element name="Руководитель" type="tns:КлючевыеСвойстваФизическоеЛицо" minOccurs="0"/>
        <xs:element name="ГлавныйБухгалтер" type="tns:КлючевыеСвойстваФизическоеЛицо" minOccurs="0"/>
        <xs:element name="БанковскийСчетОрганизации" type="tns:КлючевыеСвойстваБанковскийСчет" minOccurs="0"/>
        <xs:element name="Контрагент" type="tns:КлючевыеСвойстваКонтрагент"/>
        <xs:element name="ДанныеВзаиморасчетов" type="tns:ОбщиеСвойстваДанныеВзаиморасчетов"/>
        <xs:element name="Заказ" type="tns:КлючевыеСвойстваЗаказКлиента" minOccurs="0"/>
        <xs:element name="Комментарий" type="xs:string" minOccurs="0"/>
        <xs:element name="ТипЦен" type="tns:КлючевыеСвойстваТипЦен" minOccurs="0"/>
        <xs:element name="Налогообложение" type="tns:Налогообложение" minOccurs="0"/>
        <xs:element name="СчетУчетаРасчетовПоТаре" type="xs:string" minOccurs="0"/>
        <xs:element name="ГруппаНастроекФинансовогоУчетаРасчетов" type="tns:КлючевыеСвойстваГруппаНастроекФинансовогоУчетаРасчетов" minOccurs="0"/>
        <xs:element name="Услуги" type="tns:Документ.АктВыполненныхРабот.Услуги"/>
        <xs:element name="СкидкиНаценки" type="tns:СкидкиНаценки" minOccurs="0"/>
        <xs:element name="ДополнительныеРеквизиты" type="tns:ДополнительныеРеквизиты" minOccurs="0"/>
        <xs:any namespace="##any" maxOccurs="unbounded" minOccurs="0" processContents="lax"/>
      </xs:sequence>
      <xs:anyAttribute namespace="##any" processContents="lax"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

Первое поле документа – это «Ключевые свойства» типа «КлючевыеСвойстваАктВыполненныхРабот»:

```
<xs:complexType name="КлючевыеСвойстваАктВыполненныхРабот">
  <xs:sequence>
    <xs:element name="Ссылка" type="tns:ДокументСсылка.АктВыполненныхРабот" minOccurs="0"/>
    <xs:element name="Дата" type="xs:dateTime"/>
    <xs:element name="Номер" type="tns:ТипНомерДокумента"/>
    <xs:element name="Организация" type="tns:КлючевыеСвойстваОрганизация"/>
    <xs:any namespace="##any" maxOccurs="unbounded" minOccurs="0" processContents="lax"/>
  </xs:sequence>
  <xs:anyAttribute namespace="##any" processContents="lax"/>
</xs:complexType>
```

Первое поле «Ссылка» в типе «КлючевыеСвойстваАктВыполненныхРабот» - это элемент типа "ДокументСсылка.АктВыполненныхРабот", являющийся расширением типа «Ref»:

[Копировать в буфер обмена](#)

```
<xs:simpleType name="ДокументСсылка.АктВыполненныхРабот">
  <xs:restriction base="ns1:Ref"/>
</xs:simpleType>
```

Тип «Ref» описан в файле ExchangeMessage.xsd и формально является строкой, но, как было сказано выше, конфигурация ожидает в этом поле GUID в строковом представлении:

[Копировать в буфер обмена](#)

```
<xs:simpleType name="Ref">
  <xs:restriction base="xs:string"/>
</xs:simpleType>
```

Далее в документе «Документ.АктВыполненныхРабот» следуют поля, ссылающиеся на ответственного и валюту (оба – ссылки на соответствующие объекты в системе), сумма (тип «ТипСумма», цифровой) и т.д.

Элемент Услуги имеет тип «Документ.АктВыполненныхРабот.Услуги» – это коллекция строк типа «Документ.АктВыполненныхРабот.Услуги.Строка»:

[Копировать в буфер обмена](#)

```
<xs:complexType name="Документ.АктВыполненныхРабот.Услуги">
  <xs:sequence><xs:element name="Строка" type="tns:Документ.АктВыполненныхРабот.Услуги.Строка"
    maxOccurs="unbounded"/>
</xs:sequence></xs:complexType>
```

Тип «Документ.АктВыполненныхРабот.Услуги.Строка» описывает строку, соответствующую конкретной услуге:

[Копировать в буфер обмена](#)

```
<xs:complexType name="Документ.АктВыполненныхРабот.Услуги.Строка">
  <xs:sequence>
    <xs:element name="НомерСтрокиДокумента" type="tns:ТипНомерСтроки" minOccurs="0"/>
    <xs:element name="Номенклатура" type="tns:КлючевыеСвойстваНоменклатура"/>
    <xs:element name="Характеристика" type="tns:КлючевыеСвойстваХарактеристикаНоменклатуры" minOccurs="0"/>
    <xs:element name="Количество" type="tns:ТипКоличество"/>
    <xs:element name="Сумма" type="tns:ТипСумма"/>
    <xs:element name="Цена" type="tns:ТипСумма"/>
    <xs:element name="СтавкаНДС" type="tns:СтавкиНДС" minOccurs="0"/>
    <xs:element name="СуммаНДС" type="tns:ТипСумма" minOccurs="0"/>
    <xs:element name="Содержание" type="xs:string" minOccurs="0"/>
    <xs:element name="СчетУчетаДоходов" type="xs:string" minOccurs="0"/>
    <xs:element name="СчетУчетаРасходов" type="xs:string" minOccurs="0"/>
    <xs:any namespace="##any" maxOccurs="unbounded" minOccurs="0" processContents="lax"/>
  </xs:sequence>
  <xs:anyAttribute namespace="##any" processContents="lax"/>
</xs:complexType>
```

Как можно заметить, некоторые типы в качестве последнего элемента содержат строчку

[Копировать в буфер обмена](#)

```
<xs:any namespace="##any" maxOccurs="unbounded" minOccurs="0" processContents="lax"/>
```

Эта строка позволяет нам добавлять в элементы произвольное количество (maxOccurs="unbounded") дополнительных полей любого типа (xs:any), не нарушая XSD-схему. Это та самая ситуация, когда мы расширяем объекты новыми полями, повышая при этом минорную версию формата. В этом случае программа, которая уже умеет работать с новыми полями, сможет их обработать соответствующим образом; если же программа написана для работы с более старыми минорными версиями формата – она пропустит «незнакомые» поля, не обработав их.

Поля (элементы) сущности, у которых отсутствует атрибут minOccurs="0", являются обязательными к заполнению (согласно стандарту XSD).

Как с помощью формата EnterpriseData обмениваться данными с конфигурациями

При обмене с использованием планов обмена конфигурации в ходе синхронизации передают только информацию об изменениях, произошедших со времени последней синхронизации (чтобы минимизировать объем передаваемой информации). При первой синхронизации конфигурация выгрузит все объекты в формате EnterpriseData в XML-файл (поскольку все они являются «новыми» для стороннего приложения).

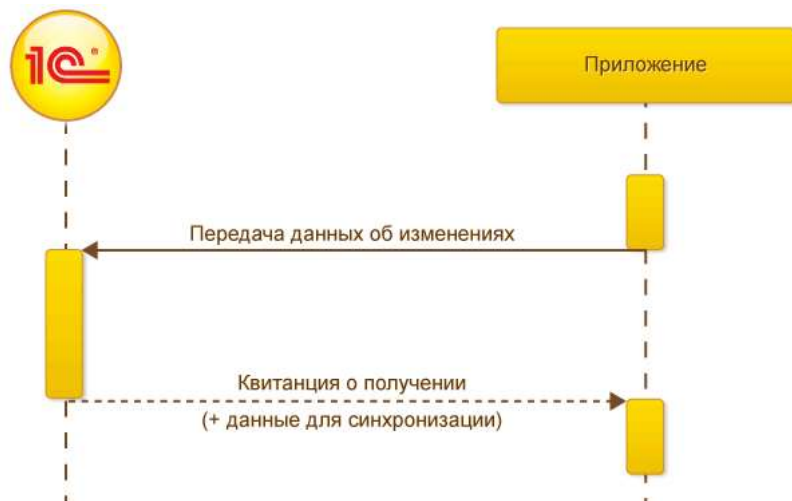
Следующий шаг за сторонним приложением – оно должно обработать информацию из XML-файла и при следующем сеансе синхронизации поместить в секцию <Confirmation> информацию, что сообщение от конфигурации за определенным номером успешно принято (поместить в поле ReceivedNo номер полученного от конфигурации сообщения). Сообщение-квитанция является для конфигурации сигналом, что все объекты успешно обработаны внешним приложением и информацию о них передавать больше не нужно. Помимо квитанции XML-файл от стороннего приложения также может содержать данные для синхронизации (в секции <Body>).



После получения сообщения-квитанции конфигурация помечает все изменения, переданные в предыдущем сообщении, как успешно синхронизированные. Лишь несинхронизированные изменения в объектах (создание новых, изменение и удаление существующих) будут отправлены во внешнее приложение при следующем сеансе синхронизации.

При передаче данных от внешнего приложения в конфигурацию картина меняется на обратную. Приложение должно заполнить секцию <Confirmation> соответствующим образом, а в секцию <Body> поместить объекты для синхронизации в формате EnterpriseData.

Конфигурация после обработки файла сформирует XML-файл, который будет содержать сообщение-квитанцию и новые данные для синхронизации со стороны конфигурации (если такие есть со времени последнего сеанса синхронизации).



Учет номеров отправленных и полученных сообщений

Для успешной работы механизма квитирования важно, чтобы стороннее приложение вело учет номеров сообщений, полученных от конфигурации и отправленных в конфигурацию (так же, как делает это сама конфигурация).

В информационной базе есть служебная таблица, в которую заносится информация о настроенных планах обмена. Интересующая нас ее часть содержит четыре поля:

1. From. Двухбуквенный код приложения – источника данных. В рассматриваемом примере у конфигурации он будет – «УП» (Управление Предприятием).
2. To. Двухбуквенный код приложения, с которым осуществляется обмен.
3. MessageNo. Целое число, номер последнего сообщения, отправленного из конфигурации в стороннее приложение.
4. ReceivedNo. Целое число, номер последнего сообщения, принятого конфигурацией от стороннего приложения.

Запись о настроенном обмене с приложением ZZ; ни одного обмена данными еще не было, поэтому в колонках номеров сообщений нули:

From	To	MessageNo	ReceivedNo
УП	ZZ	0	0

Предположим, что обмен данными между конфигурацией и приложением ZZ уже идет некоторое время и конфигурация отослала N сообщений приложению ZZ и получила от него последнее сообщение с номером M:

From	To	MessageNo	ReceivedNo
УП	ZZ	N	M

В информационную базу был введен новый документ, акт выполненных работ. Конфигурация формирует сообщение (файл с именем Message_<КодОтправителя>_<КодПолучателя>.xml, в нашем случае - Message_УП_ZZ.xml) для приложения ZZ, в секцию Body кладет документ с типом «Документ.АктВыполненныхРабот», а секцию Header формирует таким образом:

```

<Header>
  <Format>http://v8.1c.ru/edi/edi_std/EnterpriseData/1.0.beta</Format>
  <CreationDate>2015-02-05T08:01:52</CreationDate>
  <Confirmation>
    <ExchangePlan>СинхронизацияДанныхЧерезУниверсальныйФормат</ExchangePlan>
    <To>ZZ</To>
    <From>УП</From>
    <MessageNo>N+1</MessageNo>
    <ReceivedNo>M</ReceivedNo>
  </Confirmation>
  <AvailableVersion>1.0.beta</AvailableVersion>
  <AvailableVersion>1.0</AvailableVersion>
</Header>
  
```

[Копировать в буфер обмена](#)

Обновляется запись о настроенном обмене с приложением ZZ, увеличился счётчик сообщений, отправленных из конфигурации в приложение ZZ.

From	To	MessageNo	ReceivedNo
УП	ZZ	N+1	M

Получение подтверждения (квитанции) от приложения ZZ

Теперь конфигурация ожидает подтверждения от приложения ZZ, что данные были получены и успешно обработаны. Если данные обработаны успешно, приложение ZZ должно сформировать файл ответа (Message_<КодОтправителя>_<КодПолучателя>.xml, в нашем случае - Message_ZZ_УП.xml) и поместить его в настроенный канал. Секция Header выглядит так:

```

<Header>
  <Format>http://v8.1c.ru/edi/edi_std/EnterpriseData/1.0.beta</Format>
  <CreationDate>2015-02-05T08:01:52</CreationDate>
  <Confirmation>
    <ExchangePlan>СинхронизацияДанныхЧерезУниверсальныйФормат</ExchangePlan>
    <To> УП </To>
    <From> ZZ </From>
    <MessageNo>M+1</MessageNo>
    <ReceivedNo>N + 1</ReceivedNo>
  </Confirmation>
  <AvailableVersion>1.0.beta</AvailableVersion>
  <AvailableVersion>1.0</AvailableVersion>
</Header>
  
```

[Копировать в буфер обмена](#)

Этим сообщением приложение ZZ подтверждает, что данные из сообщения от конфигурации с номером N+1 получены и обработаны. Конфигурация обновляет таблицу:

From	To	MessageNo	ReceivedNo
УП	ZZ	N+1	M+1

Следует отметить, что приложение ZZ не обязано увеличивать свой счетчик отправленных сообщений именно на единицу; это может быть любое число, единственное условие – оно должно быть больше номера предыдущего сообщения, отправленного из приложения ZZ в конфигурацию.

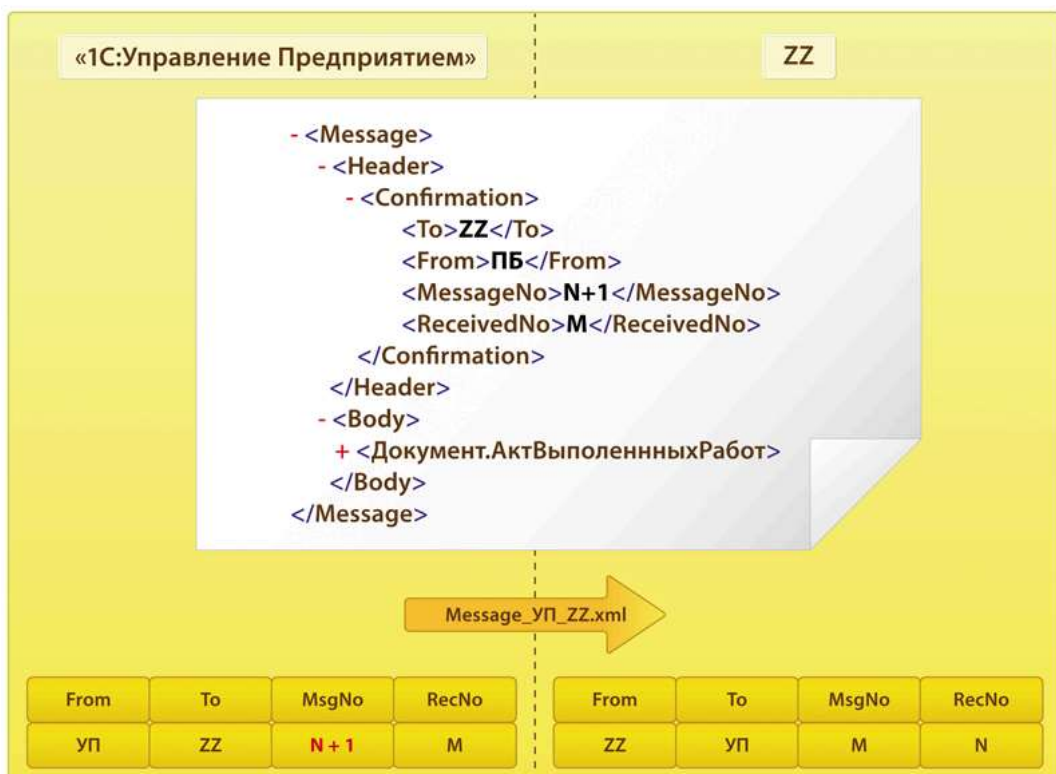
Если у приложения ZZ есть данные для синхронизации с информационной базой, оно может поместить их в секцию <Body> того же сообщения (а может и отправить отдельным сообщением, не забыв при этом увеличить номер сообщения MessageNo).

Полная картина обмена

Предположим для простоты, что в приложении ZZ для хранения информации об обмене данными с конфигурацией используется таблица той же структуры, что и в конфигурации.

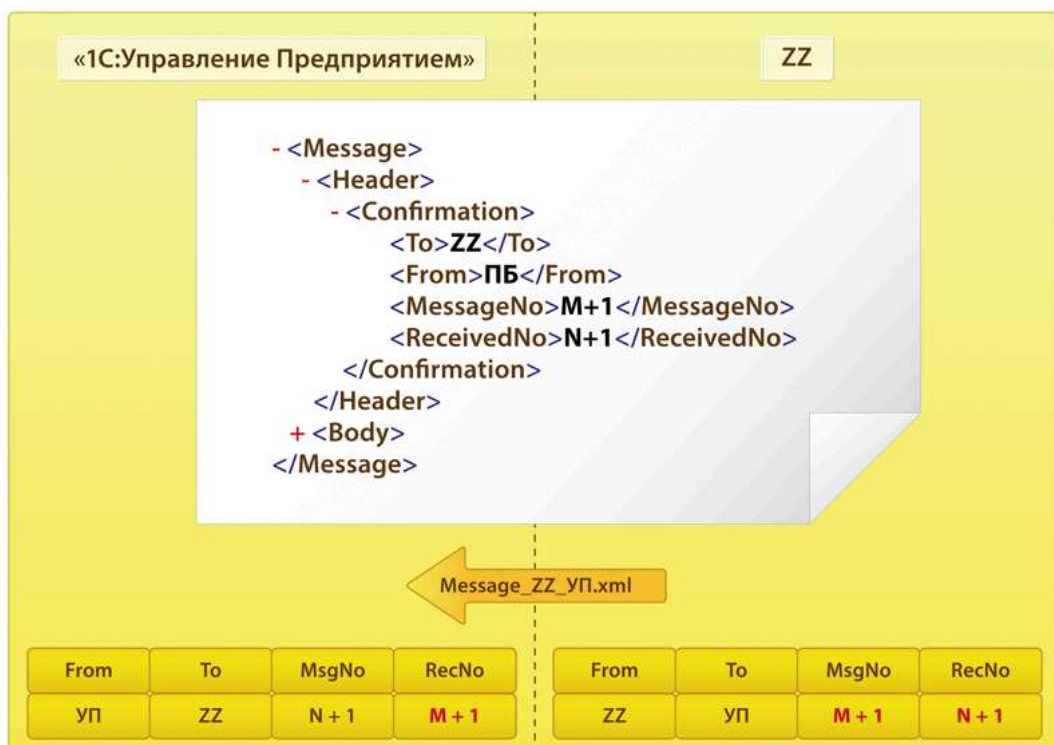
«1С:Управление Предприятием»				ZZ			
From	To	MsgNo	RecNo	From	To	MsgNo	RecNo
УП	ZZ	N	M	ZZ	УП	M	N

Обратите внимание, что в приложении ZZ в таблице с информацией об обмене данными с конфигурацией в полях MessageNo и ReceivedNo лежат цифры, зеркально отображенные из аналогичной таблицы конфигурации: приложение ZZ получило от конфигурации M сообщений и отправило в конфигурацию N сообщений.



После отправки сообщения из конфигурации в приложение ZZ в информационной базе увеличивается счетчик отправленных в приложение ZZ сообщений.

Приложение ZZ посылает сообщение номер M+1 о подтверждении получения сообщения от конфигурации с номером N+1



Приложение ZZ посылает сообщение номер M+1 о подтверждении получения сообщения от конфигурации с номером N+1

Приложение ZZ после обработки данных, полученных от конфигурации:

1. Обновляет номер последнего сообщения, полученного от конфигурации: RecievedNo = N+1
2. Формирует сообщение-квитанцию, подтверждающее, что данные сообщения из информационной базы с номером N+1 получены, присваивает квитанции номер M+1 и обновляет номер последнего сообщения, отправленного в конфигурацию: MessageNo = M+1.

Конфигурация, получив квитанцию от приложения ZZ, делает следующее:

1. Обновляет номер последнего полученного от ZZ сообщения: ReceivedNo = M+1.
2. Помечает изменения в объектах, переданные в составе сообщения «Message_УП_ZZ.xml» за номером N+1, как синхронизированные с приложением ZZ. В дальнейшем обмене сообщениями о синхронизации с приложением ZZ эти изменения фигурировать не будут.

Ресурсы

- [Описание, XSD-схемы и актуальные версии формата EnterpriseData](#)
- [Примеры использования веб-сервисов на C# и Java](#)